Backend Developer Interview Knowledge Base

AI Interview Coach ��� RAG Knowledge Base

2026-07-11

Contents

Backend Developer Interview Knowledge Base

> A structured, retrieval-friendly reference for an AI Interview Coach. Each topic is self-contained with definitions, examples, best practices, and common interview mistakes for efficient chunking and semantic search.

Backend Architecture

Definition: Backend architecture describes how server-side components (application logic, databases, APIs, caching, messaging) are organized to handle requests, process data, and return responses to clients.

Core layers in a typical backend: - **Presentation/API layer** – exposes endpoints (REST, GraphQL, gRPC) to clients. - **Business logic layer** – application rules, validation, orchestration. - **Data access layer** – interacts with databases/external services. - **Infrastructure layer** – caching, queues, logging, monitoring.

`Client → API Gateway → Service (Controller → Service → Repository) → Database`

Best Practices: Separate concerns into layers (controllers handle HTTP, services handle logic, repositories handle data); keep business logic out of controllers; design for statelessness so any server instance can handle any request.

Common Mistakes: Putting database queries directly in controllers (tight coupling, hard to test); mixing business logic across multiple layers inconsistently.

Interview Tip: Be ready to sketch a request's journey from client to database and back, naming each layer's responsibility.

REST APIs

Definition: REST (Representational State Transfer) is an architectural style for designing networked APIs using stateless HTTP requests, resource-based URLs, and standard HTTP methods.

Key principles: statelessness (no session state stored on server between requests), resource-based URLs (`/users/123`, not `/getUser?id=123`), uniform interface (standard HTTP verbs), cacheable responses where appropriate.

```
GET    /api/users          → list users
GET    /api/users/123      → get user 123
POST   /api/users          → create a user
PUT    /api/users/123      → replace user 123
PATCH  /api/users/123      → partially update user 123
DELETE /api/users/123      → delete user 123
```

Best Practices: Use nouns (not verbs) in URLs; use plural resource names; nest resources logically (`/users/123/orders`); return appropriate status codes and consistent JSON structures; support pagination for large collections (`?page=2&limit=20`).

Common Mistakes: Using verbs in URLs (`/getUsers`, `/createUser`) instead of relying on HTTP methods; returning `200 OK` for errors; ignoring statelessness by storing session data server-side without a shared store.

Interview Tip: Know the difference between REST and RPC-style APIs, and be ready to explain why REST scales well for stateless, cacheable resource

access.

HTTP Methods

Definition: Standard verbs that indicate the intended action on a resource.

Method	Purpose	Idempotent	Safe	Has Body
GET	Retrieve a resource	Yes	Yes	No
POST	Create a resource	No	No	Yes
PUT	Replace a resource entirely	Yes	No	Yes
PATCH	Partially update a resource	No*	No	Yes
DELETE	Remove a resource	Yes	No	Optional

*Idempotent means repeating the same request produces the same result; PATCH is not guaranteed idempotent depending on implementation.

Example:

```
app.put('/api/users/:id', (req, res) => {
  // Replaces the entire user resource with req.body
});
app.patch('/api/users/:id', (req, res) => {
  // Updates only the fields provided in req.body
});
```

Best Practices: Use PUT when sending the complete resource representation; use PATCH for partial updates; never use GET for state-changing operations.

Common Mistakes: Confusing PUT (full replace) with PATCH (partial update); using POST for all operations regardless of semantics.

Status Codes

Definition: Three-digit HTTP response codes indicating the result of a request, grouped by category.

Range	Category	Common Codes
2xx	Success	`200` OK, `201` Created, `204` No Content
3xx	Redirection	`301` Moved Permanently, `304` Not Modified
4xx	Client Error	`400` Bad Request, `401` Unauthorized, `403` Forbidden, `404` Not Found, `409` Conflict, `422` Unprocessable Entity, `429` Too Many Requests
5xx	Server Error	`500` Internal Server Error, `502` Bad Gateway, `503` Service Unavailable

Best Practices: Return `201 Created` (with a `Location` header) after successful resource creation; use `204 No Content` for successful deletes with no body; use `422` for validation errors versus `400` for malformed requests.

Common Mistakes: Confusing `401` (not authenticated — missing/invalid credentials) with `403` (authenticated but not authorized to access the resource); returning `200` for every response regardless of outcome, forcing clients to parse the body to detect errors.

Interview Tip: Be ready to explain `401` vs `403` with a concrete example — logging in with a bad password is `401`; a logged-in user trying to access an admin-only route is `403`.

Authentication

Definition: The process of verifying **who** a user is (identity confirmation), typically via credentials (username/password, tokens, biometrics).

Common approaches: session-based (server stores session, client holds a session cookie), token-based (JWT, client holds a signed token), OAuth (delegated authentication via a third party), API keys (for service-to-service auth).

```
// Simplified login flow
app.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  const valid = await bcrypt.compare(req.body.password, user.passwordHash)
  if (!valid) return res.status(401).json({ error: 'Invalid credentials' }
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { ex
  res.json({ token });
});
```

Best Practices: Always hash passwords with a strong algorithm (`bcrypt`, `argon2`); never store plaintext passwords; use HTTPS to protect credentials in transit; implement account lockout/rate limiting on login endpoints.

Common Mistakes: Storing passwords in plaintext or with weak hashing (MD5, SHA1 without salting); rolling custom crypto instead of using vetted libraries.

Authorization

Definition: The process of determining **what** an authenticated user is allowed to do (permissions/access control), enforced after authentication.

Common models: Role-Based Access Control (RBAC — permissions tied to roles like `admin`, `user`), Attribute-Based Access Control (ABAC — permissions based on resource/user attributes), Access Control Lists (ACL).

```
function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}
app.delete('/api/users/:id', authenticate, requireRole('admin'), deleteUse
```

Best Practices: Enforce authorization checks on the server for every protected route — never trust client-side checks alone; apply the principle of least privilege.

Common Mistakes: Confusing authentication with authorization (they are distinct concerns); performing authorization checks only in the UI, allowing direct API calls to bypass them.

Interview Tip: Be ready to state the one-line distinction: *authentication = who you are; authorization = what you can do.*

JWT (JSON Web Token)

Definition: A compact, self-contained, signed token format used to securely transmit claims (user identity, permissions) between parties. Structure: `header.payload.signature`, Base64Url-encoded.

```
const jwt = require('jsonwebtoken');

// Signing
const token = jwt.sign({ userId: 123, role: 'admin' }, process.env.JWT_SEC

// Verifying (middleware)
function authenticate(req, res, next) {
```

```
  const token = req.headers.authorization?.split(' ')[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

Key properties: stateless (server doesn't store sessions — the token itself carries the claims); signed (not encrypted by default — anyone can decode the payload, but cannot alter it without invalidating the signature).

Best Practices: Keep JWTs short-lived (e.g., 15 min–1 hr) and pair with refresh tokens for longer sessions; store JWTs in `HttpOnly` cookies (not `localStorage`) to reduce XSS risk; never put sensitive data (passwords) in the payload since it's readable, not encrypted.

Common Mistakes: Assuming JWTs are encrypted (they're only signed/encoded — Base64 is trivially decodable); storing JWTs in `localStorage` (vulnerable to XSS token theft); using a weak or hardcoded signing secret.

Interview Tip: Be ready to explain how a server "trusts" a stateless JWT — via signature verification using a secret/public key, without needing a database lookup for every request.

OAuth

Definition: An open standard for **delegated authorization** — allowing a third-party application to access a user's resources on another service without exposing the user's credentials (e.g., "Sign in with Google").

Key roles: Resource Owner (the user), Client (the app requesting access), Authorization Server (issues tokens), Resource Server (hosts the protected data).

Authorization Code flow (simplified):

1. Client redirects user to Authorization Server login/consent screen.
2. User approves → Authorization Server redirects back with an authorization c
3. Client exchanges the code (+ client secret) for an access token (server-to-server).
4. Client uses the access token to call the Resource Server's API.

Best Practices: Use the Authorization Code flow with PKCE for public clients (SPAs, mobile apps); never expose client secrets in frontend code; validate the `state` parameter to prevent CSRF during the OAuth flow.

Common Mistakes: Confusing OAuth (authorization framework) with OpenID Connect (authentication layer built on top of OAuth 2.0, used for identity/login); implementing the Implicit flow (deprecated, less secure) instead of Authorization Code + PKCE.

Interview Tip: Clarify: OAuth answers "can this app access my data?" while authentication (e.g., OpenID Connect) answers "who is this user?" — OAuth alone is not an authentication protocol.

MVC Architecture

Definition: Model-View-Controller is a design pattern separating an application into three interconnected components: **Model** (data/business logic), **View** (presentation, often the client or template), **Controller** (handles requests, coordinates Model and View).

```
// Controller
app.get('/users/:id', async (req, res) => {
  const user = await UserModel.findById(req.params.id); // Model
  res.json(user); // View (JSON representation for an API)
});
```

Best Practices: Keep controllers thin (delegate business logic to services); keep models focused on data structure/persistence; in APIs, the "View" is

often just a serialized JSON response rather than a rendered template.

Common Mistakes: Putting business logic directly in controllers, making them bloated and hard to test; tightly coupling the Model to HTTP-specific concerns (request/response objects).

Interview Tip: In modern API backends, many teams extend MVC to MVCS (Model-View-Controller-Service) — be ready to explain why the service layer was added (to isolate reusable business logic from HTTP handling).

Microservices

Definition: An architectural style where an application is composed of small, independently deployable services, each responsible for a specific business capability, communicating over the network (HTTP, gRPC, message queues).

Characteristics: independent deployment, own database per service (data ownership), communication via APIs/events, technology heterogeneity possible per service.

```
API Gateway → [Auth Service] [Order Service] [Payment Service] [Inventory Serv
                    ↓               ↓               ↓               ↓
              Auth DB       Order DB      Payment DB     Inventory DB
```

Best Practices: Design services around bounded contexts (Domain-Driven Design); use asynchronous messaging (queues/event buses) for cross-service communication where possible to reduce coupling; implement centralized logging/tracing (correlation IDs) since requests span multiple services.

Common Mistakes: Creating a “distributed monolith” — services that are technically separate but tightly coupled and must be deployed together; sharing a single database across services (breaks independence); underestimating the operational complexity (network latency, partial failures, distributed transactions).

Interview Tip: Be ready to discuss trade-offs: microservices improve scalability and team autonomy but add complexity in deployment, monitoring, and data consistency (eventual consistency, sagas) compared to a monolith.

Monolithic Architecture

Definition: An architectural style where the entire application (UI, business logic, data access) is built and deployed as a single unit/codebase.

Best Practices: Structure the codebase with clear internal module boundaries (even in a monolith) to ease a future transition to microservices if needed; use a monolith for smaller teams/early-stage products where operational simplicity matters more than independent scaling.

Common Mistakes: Assuming microservices are always superior — for small teams/simple domains, a monolith is often faster to build, test, and deploy, and avoids premature distributed-systems complexity.

Interview Tip: A frequent question: *"When would you choose a monolith over microservices?"* — Answer: smaller teams, early-stage products, simpler operational needs, and when the domain boundaries aren't yet well understood (avoid premature decomposition).

Node.js Overview

Definition: Node.js is a JavaScript runtime built on Chrome's V8 engine, enabling server-side JavaScript execution with a non-blocking, event-driven, single-threaded architecture (using an event loop for concurrency).

Key characteristics: single-threaded event loop for I/O-bound concurrency; non-blocking asynchronous I/O (file system, network); npm ecosystem for package management; well-suited for I/O-heavy workloads (APIs, real-time apps) less ideal for CPU-heavy tasks (blocks the event loop).

```
const fs = require('fs');
fs.readFile('data.txt', 'utf8', (err, data) => {
  console.log(data); // non-blocking — other code continues executing mean
});
console.log('This runs before the file read completes');
```

Best Practices: Offload CPU-intensive work to worker threads or separate services to avoid blocking the event loop; use asynchronous APIs (`fs.promises`, `async/await`) over synchronous ones (`fs.readFileSync`) in request-handling code.

Common Mistakes: Using synchronous/blocking calls in request handlers, freezing the entire server for all concurrent requests; assuming Node.js is multi-threaded by default (it uses a single main thread for JS execution, with libuv handling I/O concurrency behind the scenes).

Interview Tip: Be ready to explain why Node.js is well-suited for I/O-bound APIs but a poor fit for CPU-bound tasks (e.g., image processing) without offloading to worker threads or a separate service.

Express.js Basics

Definition: Express.js is a minimal, unopinionated web framework for Node.js used to build APIs and web servers, providing routing, middleware support, and HTTP utility methods.

```
const express = require('express');
const app = express();
app.use(express.json()); // parse JSON request bodies

app.get('/api/health', (req, res) => {
  res.status(200).json({ status: 'ok' });
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

Best Practices: Organize routes into separate router modules (`express.Router()`) rather than defining all routes in one file; use middleware for cross-cutting concerns (logging, auth, validation).

Common Mistakes: Putting all logic in a single monolithic `app.js` file as the project grows; forgetting `express.json()` middleware, causing `req.body` to be `undefined`.

Interview Tip: Be ready to build a small CRUD route on the spot — a very common live-coding request.

Middleware

Definition: Functions that execute during the request-response cycle, with access to `req`, `res`, and a `next()` function to pass control to the next middleware/handler. Used for cross-cutting concerns like logging, authentication, and error handling.

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // must call next() to continue the chain
}
app.use(logger);

// Error-handling middleware (4 arguments signals Express to treat it as e
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong' });
});
```

Middleware types: application-level (`app.use`), router-level, error-handling (4 params), built-in (`express.json()`), third-party (`cors`, `helmet`, `morgan`).

Best Practices: Keep middleware focused on a single concern; place error-handling middleware **last**, after all routes; always call `next()` (or send a response) to avoid hanging requests.

Common Mistakes: Forgetting to call `next()`, causing the request to hang indefinitely; placing error-handling middleware before regular routes (it won't catch their errors).

Interview Tip: Be ready to explain middleware execution order — Express processes middleware top-to-bottom in the order they're registered.

Routing

Definition: The mechanism that maps HTTP request URLs and methods to specific handler functions.

```
const router = express.Router();

router.get('/', getAllUsers);
router.get('/:id', getUserById);
router.post('/', createUser);
router.put('/:id', updateUser);
router.delete('/:id', deleteUser);

app.use('/api/users', router); // mount router with a base path
```

Best Practices: Group related routes into dedicated router modules by resource; use route parameters (`:id`) for resource identifiers and query parameters (`?sort=asc`) for filtering/sorting/pagination; version routes when introducing breaking changes (`/api/v1/users`).

Common Mistakes: Defining overly generic catch-all routes that unintentionally shadow more specific ones (route order matters — more specific routes should be defined before generic/wildcard ones).

Environment Variables

Definition: Configuration values (API keys, database URLs, secrets, ports) stored outside the codebase, loaded at runtime, allowing different configuration per environment (development, staging, production) without code changes.

```
// .env file (never committed to version control)
// DATABASE_URL=postgres://user:pass@localhost:5432/mydb
// JWT_SECRET=supersecretvalue

require('dotenv').config();
const dbUrl = process.env.DATABASE_URL;
const port = process.env.PORT || 3000;
```

Best Practices: Never commit `.env` files or secrets to version control (add to `.gitignore`); provide a `.env.example` template documenting required variables without real values; validate required environment variables at startup and fail fast if missing.

Common Mistakes: Hardcoding secrets/config directly in source code; committing `.env` files with real credentials to a public repository (a very common real-world security incident).

Interview Tip: Be ready to explain how environment-specific configuration supports the "twelve-factor app" principle of strict separation between config and code.

Error Handling

Definition: The practice of anticipating, catching, and gracefully responding to runtime errors (validation failures, database errors, unexpected exceptions) instead of letting them crash the server or expose internals.

```
// Centralized error-handling middleware
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

app.get('/api/users/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) throw new AppError('User not found', 404);
    res.json(user);
  } catch (err) {
    next(err); // delegate to error-handling middleware
  }
});

app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({ error: err.message || 'Internal Server Error'
});
```

Best Practices: Use a centralized error-handling middleware instead of duplicating try/catch response logic in every route; distinguish operational errors (expected, e.g., validation) from programmer errors (bugs); never leak stack traces or internal details to clients in production.

Common Mistakes: Swallowing errors silently (empty `catch` blocks); returning inconsistent error response shapes across different endpoints; crashing the entire process on an unhandled promise rejection without graceful recovery/logging.

Interview Tip: Be ready to explain the difference between handling errors gracefully (returning a proper 4xx/5xx response) versus letting the pro-

cess crash — and how to catch unhandled rejections at the process level (`process.on('unhandledRejection', ...)`) as a safety net.

Logging

Definition: Recording application events (requests, errors, state changes) to help with debugging, monitoring, and auditing in production.

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [new winston.transports.Console(), new winston.transports.Fi
});

logger.info('Server started', { port: 3000 });
logger.error('Database connection failed', { error: err.message });
```

Best Practices: Use structured logging (JSON) for easier querying/aggregation in log management tools (ELK, Datadog); include contextual data (request ID, user ID) for traceability; use appropriate log levels (`debug`, `info`, `warn`, `error`); never log sensitive data (passwords, tokens, PII).

Common Mistakes: Using `console.log` everywhere in production code instead of a proper logging library with levels/transports; logging sensitive information in plaintext.

Interview Tip: Be ready to explain the value of correlation/request IDs for tracing a single request across multiple microservices in distributed systems.

Validation

Definition: Verifying that incoming data (request bodies, query params) meets expected structure, types, and constraints before processing, to prevent invalid data and security vulnerabilities.

```
const { body, validationResult } = require('express-validator');

app.post('/api/users',
  body('email').isEmail().withMessage('Invalid email'),
  body('password').isLength({ min: 8 }).withMessage('Password too short'),
  (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(422).json({ errors: errors.array() });
    }
    // proceed with valid data
  }
);
```

Best Practices: Validate on the server even if client-side validation exists (never trust the client); use schema validation libraries (`Joi`, `Zod`, `express-validator`) rather than ad-hoc manual checks; validate early, before touching the database.

Common Mistakes: Relying solely on client-side validation (trivially bypassed via direct API calls); inconsistent or missing validation across endpoints, leading to malformed data reaching the database.

Databases

Definition: Systems for storing, organizing, and retrieving structured or unstructured data, broadly categorized as relational (SQL) or non-relational (NoSQL).

Best Practices: Choose the database type based on data shape and access patterns, not familiarity alone; use connection pooling to avoid exhausting database connections under load; index frequently queried columns.

Common Mistakes: Choosing a database technology purely by popularity without evaluating consistency, scalability, and query pattern needs.

Interview Tip: Be ready to justify a database choice for a given scenario (e.g., "would you use SQL or NoSQL for an e-commerce order system?").

SQL

Definition: Structured Query Language — used with relational databases (PostgreSQL, MySQL) that store data in structured tables with predefined schemas and relationships (foreign keys), and support ACID transactions.

```sql
SELECT u.name, o.total
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE o.total > 100
ORDER BY o.total DESC
LIMIT 10;

INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');
UPDATE users SET email = 'new@example.com' WHERE id = 1;
DELETE FROM users WHERE id = 1;
```

ACID properties: Atomicity (all-or-nothing transactions), Consistency (valid state transitions), Isolation (concurrent transactions don't interfere), Durability (committed data persists).

Best Practices: Use parameterized queries/prepared statements to prevent SQL injection; normalize schemas to reduce redundancy, but denormalize selectively for read-heavy performance needs; index columns used in `WHERE`/`JOIN`/`ORDER BY`.

Common Mistakes: String-concatenating user input directly into SQL queries (critical SQL injection vulnerability); over-normalizing to the point of requiring excessive joins for common queries.

Interview Tip: Be ready to explain SQL injection with an example and how parameterized queries prevent it:

```
// Vulnerable:
db.query(`SELECT * FROM users WHERE email = '${email}'`);
// Safe:
db.query('SELECT * FROM users WHERE email = ?', [email]);
```

NoSQL

Definition: Non-relational databases that store data in flexible formats (documents, key-value, wide-column, graph) rather than fixed tables — optimized for scalability and schema flexibility over strict relational consistency.

Types: Document (MongoDB — JSON-like documents), Key-Value (Redis, DynamoDB), Wide-Column (Cassandra), Graph (Neo4j).

```
// MongoDB example
db.users.insertOne({ name: 'Alice', email: 'alice@example.com', tags: ['ad
db.users.find({ tags: 'admin' });
```

Best Practices: Use NoSQL for flexible/evolving schemas, high write throughput, or horizontal scaling needs; design document structure around access patterns (denormalize/embed related data when read together frequently).

Common Mistakes: Using NoSQL for highly relational data requiring complex joins and strong transactional consistency across entities; ignoring eventual consistency implications in distributed NoSQL systems.

Interview Tip: Be ready to compare SQL vs NoSQL trade-offs: SQL offers strong consistency and relational integrity; NoSQL offers flexible schemas

and easier horizontal scaling — the choice depends on data shape and consistency requirements (CAP theorem trade-offs).

ORM

Definition: Object-Relational Mapping — a library that lets developers interact with a relational database using object-oriented code instead of writing raw SQL (e.g., Sequelize, TypeORM, Prisma for Node.js; Hibernate for Java).

```
// Prisma example
const user = await prisma.user.create({
  data: { name: 'Alice', email: 'alice@example.com' }
});
const users = await prisma.user.findMany({ where: { active: true } });
```

Best Practices: Use an ORM for productivity and protection against SQL injection (parameterization is handled internally); drop down to raw queries for complex, performance-critical queries the ORM can't express efficiently.

Common Mistakes: The "N+1 query problem" — fetching a list of records and then issuing a separate query per record for related data instead of using eager loading/joins:

```
// N+1 problem: one query per user to fetch their orders
users.forEach(async user => { user.orders = await Order.find({ userId: use
// Fix: eager load with a single join query
const users = await User.findAll({ include: Order });
```

Interview Tip: The N+1 query problem is a very common ORM interview question — be ready to explain both the problem and the fix (eager loading/joins/batching).

CRUD

Definition: Create, Read, Update, Delete — the four basic operations for persistent data, typically mapped to HTTP methods (`POST`, `GET`, `PUT/PATCH`, `DELETE`) in REST APIs.

```
router.post('/users', createUser);      // Create
router.get('/users/:id', getUser);      // Read
router.put('/users/:id', updateUser);   // Update
router.delete('/users/:id', deleteUser); // Delete
```

Best Practices: Ensure each CRUD operation returns the appropriate status code (`201` for create, `200`/`204` for update/delete); wrap multi-step writes in database transactions to maintain consistency.

Common Mistakes: Not using transactions for operations that modify multiple related records, risking partial/inconsistent writes if one step fails.

API Security

Definition: Practices and mechanisms to protect APIs from unauthorized access, data breaches, and abuse.

Key measures: HTTPS everywhere (encrypt data in transit); input validation/sanitization (prevent injection attacks); authentication + authorization on every protected route; rate limiting (prevent abuse/DoS); security headers via middleware (`helmet` in Express); CORS configuration (restrict allowed origins); secrets management (environment variables, vaults — never hard-coded).

```
const helmet = require('helmet');
const cors = require('cors');

app.use(helmet());  // sets secure HTTP headers
app.use(cors({ origin: 'https://trusted-frontend.com', credentials: true }
```

Best Practices: Follow the principle of least privilege for all credentials/roles; keep dependencies updated to patch known vulnerabilities; log and monitor for suspicious activity; sanitize all user input server-side regardless of client-side checks.

Common Mistakes: Trusting client-side validation alone; exposing detailed error messages/stack traces in production responses; using wildcard CORS (*) with credentialed requests.

Interview Tip: Be ready to name the OWASP Top 10 categories relevant to APIs (e.g., broken authentication, injection, broken access control, security misconfiguration) even at a high level.

Rate Limiting

Definition: Restricting the number of requests a client can make to an API within a given time window, to prevent abuse, protect infrastructure, and ensure fair usage.

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,                 // limit each IP to 100 requests per window
  message: 'Too many requests, please try again later.'
});
app.use('/api/', limiter);
```

Common algorithms: Fixed window, Sliding window, Token bucket, Leaky bucket.

Best Practices: Apply stricter limits on sensitive endpoints (login, password reset) to mitigate brute-force attacks; return 429 `Too Many Requests` with a `Retry-After` header when limits are exceeded; use a shared store (Redis) for rate limiting across multiple server instances.

Common Mistakes: Implementing rate limiting with in-memory storage in a multi-instance deployment (each instance tracks limits independently, making the limit ineffective) instead of a shared/distributed store.

Caching

Definition: Temporarily storing frequently accessed data in a fast-access layer (memory) to reduce latency and database/computation load on repeated requests.

Levels: client-side (browser cache, `Cache-Control` headers), CDN caching (static assets), application-level caching (in-memory or Redis), database query caching.

```
app.get('/api/products/:id', async (req, res) => {
  const cacheKey = `product:${req.params.id}`;
  const cached = await redisClient.get(cacheKey);
  if (cached) return res.json(JSON.parse(cached));

  const product = await Product.findById(req.params.id);
  await redisClient.setEx(cacheKey, 3600, JSON.stringify(product)); // cac
  res.json(product);
});
```

Cache invalidation strategies: TTL (time-based expiry), write-through (update cache on write), cache-aside/lazy loading (populate cache on read miss, as shown above).

Best Practices: Cache read-heavy, infrequently-changing data; always set a reasonable TTL to avoid serving stale data indefinitely; invalidate/update cache explicitly on writes when strong freshness matters.

Common Mistakes: Caching highly volatile data without proper invalidation, serving stale results; the classic quote applies here — “There are only two hard things in Computer Science: cache invalidation and naming things.”

Interview Tip: Be ready to explain cache-aside vs write-through strategies and when each is appropriate.

Redis

Definition: An in-memory, key-value data store used for caching, session storage, rate limiting, pub/sub messaging, and as a lightweight message broker — valued for its speed (sub-millisecond latency).

```
const redis = require('redis');
const client = redis.createClient();

await client.set('user:123', JSON.stringify({ name: 'Alice' }), { EX: 3600
const value = await client.get('user:123');

await client.lPush('queue:jobs', JSON.stringify({ task: 'sendEmail' })); /
await client.expire('user:123', 60); // set/update TTL
```

Common use cases: caching, session store (shared across multiple server instances), rate limiting counters, real-time leaderboards (sorted sets), pub/sub for real-time features.

Best Practices: Set expiration (EX) on cache keys to avoid unbounded memory growth; use Redis as a shared session store in horizontally-scaled Node.js apps instead of in-memory sessions.

Common Mistakes: Using Redis as a primary/durable data store for critical data without proper persistence configuration (RDB/AOF); storing unbounded, ever-growing data without eviction policies.

Docker Basics

Definition: Docker packages an application and its dependencies into a portable, isolated **container** that runs consistently across environments (development, staging, production).

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

```
docker build -t my-app .
docker run -p 3000:3000 -e NODE_ENV=production my-app
```

Key concepts: **Image** (immutable snapshot/blueprint), **Container** (running instance of an image), **Dockerfile** (instructions to build an image), **docker-compose** (define/run multi-container applications, e.g., app + database + Redis together).

Best Practices: Use small base images (`alpine` variants) to reduce image size; use multi-stage builds to keep production images lean (exclude dev dependencies/build tools); never bake secrets into images — pass them via environment variables at runtime.

Common Mistakes: Including `node_modules`/build artifacts in version control and copying them into the image instead of installing fresh; running containers as the root user unnecessarily (security risk).

Interview Tip: Be ready to explain the difference between an image and a container (class vs instance analogy) and why containers ensure “it works on my machine” consistency across environments.

Deployment Concepts

Definition: The process and strategies for releasing application code to production environments reliably and with minimal downtime.

Common strategies: **Blue-Green deployment** (two identical environments, switch traffic instantly between them), **Rolling deployment** (gradually replace old instances with new ones), **Canary deployment** (release to a small subset of users first, monitor, then roll out fully).

CI/CD: Continuous Integration (automatically build/test code on every commit) + Continuous Deployment/Delivery (automatically deploy passing builds to production/staging).

Best Practices: Automate deployments via CI/CD pipelines to reduce human error; use health checks so load balancers/orchestrators only route traffic to healthy instances; always have a rollback plan.

Common Mistakes: Deploying manually without automated tests/health checks, increasing the risk of shipping broken code to production; no rollback strategy when a deployment fails.

Load Balancing

Definition: Distributing incoming network traffic across multiple server instances to improve availability, reliability, and horizontal scalability.

Common algorithms: Round Robin (requests distributed sequentially), Least Connections (routes to the server with fewest active connections), IP Hash (routes based on client IP for session stickiness).

```
Client → Load Balancer → [Server 1] [Server 2] [Server 3]
```

Best Practices: Design backend services to be stateless so any instance can handle any request (store session state in a shared store like Redis, not in-memory); use health checks to automatically remove unhealthy instances from rotation.

Common Mistakes: Relying on server-side in-memory session state in a load-balanced environment, causing inconsistent behavior when a user's requests hit different servers ("session affinity" issues).

Interview Tip: Be ready to explain why statelessness is essential for effective load balancing and horizontal scaling.

Scalability

Definition: A system's ability to handle increased load (more users, more data, more requests) by adding resources.

Vertical scaling (scale up): adding more power (CPU/RAM) to an existing server — simple but has a hard ceiling and a single point of failure. **Horizontal scaling (scale out):** adding more server instances behind a load balancer — more complex but virtually unlimited and more resilient.

Best Practices: Design for horizontal scalability from the start where growth is expected (stateless services, externalized session/cache state); scale the database independently via read replicas, sharding, or caching layers, since it's often the bottleneck before application servers are.

Common Mistakes: Assuming scaling application servers alone solves performance issues while ignoring the database as the actual bottleneck; premature scaling investment before establishing real load requirements.

Interview Tip: Be ready to discuss how you'd scale a specific system component (e.g., "how would you scale a read-heavy API?") — typical answers include caching, read replicas, and CDN usage.

File Upload

Definition: Handling binary file data (images, documents) sent from clients to the server, typically via `multipart/form-data` encoding.

```
const multer = require('multer');
const upload = multer({
  dest: 'uploads/',
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit
  fileFilter: (req, file, cb) => {
    const allowed = ['image/png', 'image/jpeg'];
    cb(null, allowed.includes(file.mimetype));
  }
});

app.post('/api/upload', upload.single('file'), (req, res) => {
  res.json({ filename: req.file.filename });
});
```

Best Practices: Validate file type and size on the server (never trust the client's declared MIME type alone — verify actual content when security matters); store uploaded files in object storage (S3, Cloud Storage) rather than the application server's local disk in production (ephemeral/non-scalable); generate unique filenames to avoid collisions/overwrites.

Common Mistakes: Not limiting file size, allowing denial-of-service via huge uploads; trusting the client-supplied file extension/MIME type without validation, allowing malicious file uploads (e.g., disguised executables).

Interview Tip: Be ready to explain why uploaded files shouldn't be stored on a single server's local disk in a horizontally-scaled, load-balanced architecture (files wouldn't be accessible from other instances) — object storage or a shared file service solves this.

API Versioning

Definition: Strategies for evolving an API over time without breaking existing client integrations.

Common approaches: - **URI versioning:** `/api/v1/users`, `/api/v2/users` — simplest, most visible. - **Header versioning:** `Accept: application/vnd.myapi.v2+json` — cleaner URLs, less discoverable. - **Query parameter versioning:** `/api/users?version=2` — simple but easy to overlook.

```
app.use('/api/v1/users', usersV1Router);
app.use('/api/v2/users', usersV2Router);
```

Best Practices: Version APIs from the start of any public-facing API to avoid painful retrofits later; maintain backward compatibility within a version (only introduce breaking changes in a new version); communicate deprecation timelines clearly to API consumers.

Common Mistakes: Making breaking changes to an existing API version without warning, breaking client integrations; over-versioning trivial, non-breaking changes unnecessarily.

Interview Tip: Be ready to explain what counts as a "breaking change" (removing/renaming fields, changing data types, altering required parameters) versus a safe additive change (adding new optional fields).

Best Practices (Summary)

- Design stateless, horizontally scalable services; externalize session state (Redis) when load balancing.
- Validate and sanitize all input server-side, regardless of client-side checks.
- Use parameterized queries/ORMs to prevent SQL injection.
- Hash passwords with `bcrypt`/`argon2`; never store plaintext credentials.
- Return consistent, meaningful HTTP status codes and error response shapes.
- Use environment variables for configuration/secrets; never commit them to version control.
- Implement centralized error handling and structured logging.

- Apply rate limiting and security headers (`helmet`) to all public APIs.
- Cache read-heavy data with clear invalidation strategy (TTL or explicit invalidation).
- Write automated tests and use CI/CD pipelines for reliable, repeatable deployments.

Performance Optimization

Definition: Techniques to reduce latency, increase throughput, and use server resources efficiently under load.

Key techniques: - **Caching** at multiple layers (CDN, application, database query cache). - **Database indexing** on frequently queried/joined columns. - **Connection pooling** to reuse database connections instead of opening new ones per request. - **Pagination** for large result sets instead of returning entire datasets. - **Asynchronous/non-blocking I/O** to maximize throughput per server instance. - **Compression** (gzip/Brotli) for HTTP responses. - **Load testing** (e.g., k6, Apache JMeter) to identify bottlenecks before they hit production.

```
// Example: pagination to avoid loading entire collections
app.get('/api/products', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 20;
  const products = await Product.find()
    .skip((page - 1) * limit)
    .limit(limit);
  res.json(products);
});
```

Best Practices: Profile and measure before optimizing (avoid premature optimization); optimize the biggest bottleneck first (often the database, not the application code); use indexes deliberately (too many indexes slow down writes).

Common Mistakes: Optimizing code without profiling first, wasting effort on parts that aren't actual bottlenecks; missing database indexes on frequently filtered/joined columns, causing full table scans; N+1 query patterns silently degrading performance under load.

Interview Tip: Be ready to walk through how you would diagnose a "slow API endpoint" — check database query performance/indexes first, then look at N+1 queries, then caching opportunities, then application-level bottlenecks.

Common Interview Mistakes (Cross-Cutting)

1. **Confusing authentication and authorization** — unable to clearly state the distinction with an example.
2. **Confusing `401` and `403`** status codes.
3. **Not knowing the difference between `PUT` and `PATCH`** (full replace vs partial update).
4. **Storing JWTs insecurely** (`localStorage` instead of `HttpOnly` cookies) or assuming JWTs are encrypted.
5. **Not validating input server-side**, relying only on frontend validation.
6. **SQL injection vulnerabilities** from string-concatenated queries instead of parameterized queries.
7. **Ignoring the N+1 query problem** when discussing ORM usage.
8. **Assuming microservices are always the better choice** without discussing trade-offs (operational complexity, data consistency).
9. **Not knowing how to scale a stateful service** — missing the connection between statelessness and horizontal scaling/load balancing.
10. **Hardcoding secrets/config** instead of using environment variables.
11. **Not having a clear caching invalidation strategy** when discussing performance improvements.
12. **Overlooking rate limiting and basic API security measures** (CORS, helmet, HTTPS) when designing a "production-ready" API.

Interview Tip: When asked an open-ended system design question, struc-

ture your answer: clarify requirements → outline high-level architecture → discuss data model → discuss scaling/security/failure handling — this structured approach signals strong engineering communication skills.

End of Knowledge Base.